

一、计算机的基础

1.1 计算机基础

1.1.1 计算机基础概述

Java是一门编程语言，语言有英语、法语、俄罗斯语等，仅我们国家就有汉语、藏语、满语等，但不管是什么语言，都是用来沟通的介质。其表达意思的本质都是一样的，程序员编程的本质就是让计算机去工作，而编程语言就是程序员与计算机沟通的介质程序员要想让计算机工作，必须知道计算机能干什么，怎么干的，这也就是我们必须学习计算机基础的原因；

程序员用编程语言写程序，最终开发出的结果就是一个软件，这些软件必须运行在**操作系统**之上，你肯定会问：为何要有操作系统呢？

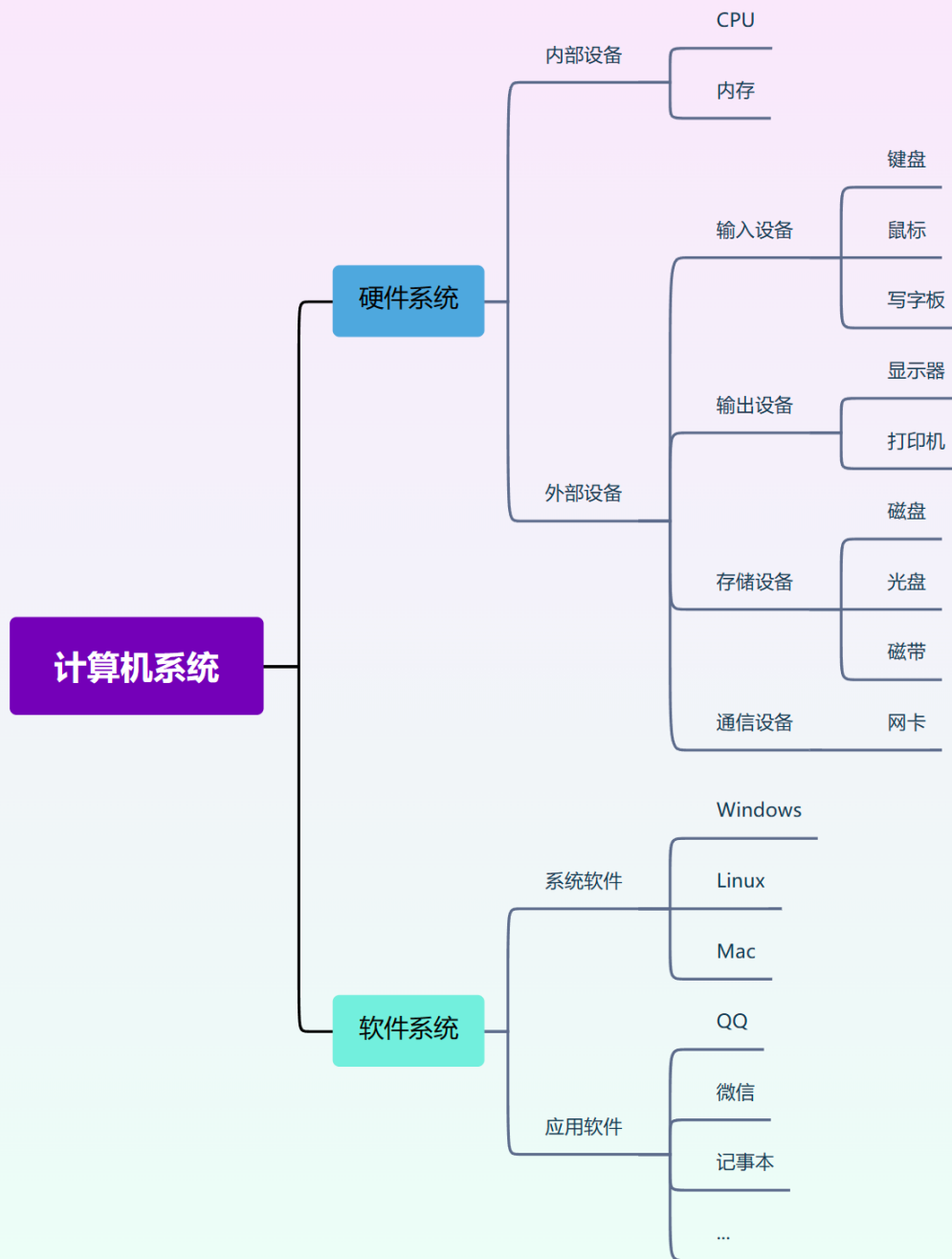
没错，远古时代的程序员确实是在没有操作系统的环境下，用编程语言直接操作硬件来编程的，你可能觉得这没有问题，但其实问题是相当严重的，因为此时你必须掌握如何操作硬件的所有具体细节，比如如何具体操作硬盘（现在你得把硬盘拆开，然后你能看见的所有东西，你都得研究明白，因为你编程时要用到它），这就严重影响了开发的效率；

Tips：操作系统的出现就是运行于硬件之上，来控制硬件的，我们开发时，只需要调用操作系统为我们提供的简单而优雅的接口就可以了；

所以要运行一个我们开发的软件需要有**硬件的支持、操作系统的支持、编写成功的软件**；其中软件是运行在操作系统之上的；

1.1.2 计算机系统简介

计算机系统分为**硬件系统**和**软件系统**两大类，如图所示：

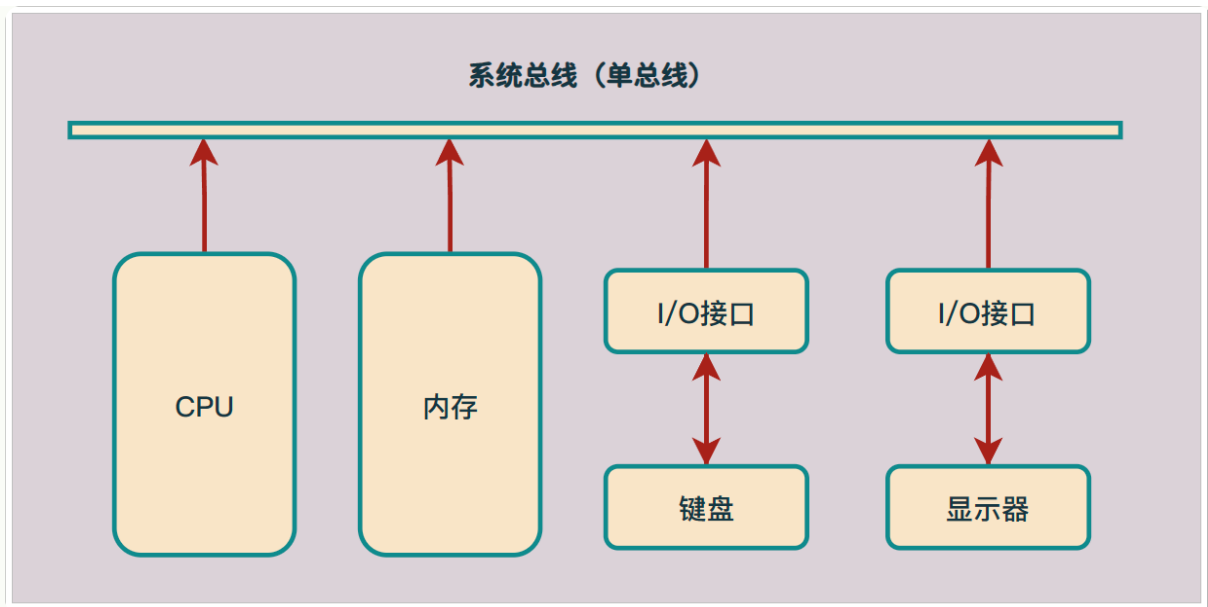


1.1.3 计算机硬件组成

计算机硬件有五大部分：

- 1) **控制器**：计算机的指挥系统。控制器通过地址访问存储器，从存储器中取出指令，经译码器分析后，根据指令分析结果产生相应的操作控制信号作用于其他部件，使得各部件在控制器控制下有条不紊地协调工作。
- 2) **运算器**：实现算术运算和逻辑运算的部件。
- 3) **存储器**：是计算机用来存放所有数据和程序的记忆部件。它的基本功能是按指定的地址存（写）入或者取（读）出信息。计算机中的存储器可分成两大类：一类是**内存储器**，简称**内存**或**主存**；另一类是**外存储器**（辅助存储器），简称**外存**或**辅存**。存储器由若干个存储单元组成，每个存储单元都有一个地址，计算机通过地址对存储单元进行读写。一个存储器所包含的字节数称为存储容量，单位有B、KB、MB、GB、TB等。
- 4) **输入设备**：是向计算机中输入信息（程序、数据、声音、文字、图形、图像等）的设备。常见的输入设备有：键盘、鼠标、图形扫描仪、触摸屏、条形码输入器、光笔等。外存储器也是一种输入设备。
- 5) **输出设备**：主要有显示器、打印机和绘图仪等。外存储器也当作一种输出设备。

控制器+运算器=CPU，**CPU、内存(主存储器)以及其他I/O设备都由一条系统总线（bus）连接起来并通过总线与其他设备通信**



- CPU：类似于人的大脑，负责控制全身和运算
- 主存：主存（内存）是人的记忆，负责临时存储
- 输入设备（Input）：是耳朵或眼睛或嘴巴，负责接收外部的信息存入内存，如键盘、鼠标、话筒、写字板等用于接收外部信息指令的设备
- 输出设备（Output）：是你的脸部（表情）或者屁股，负责经过处理后输出的结果，如显示器、打印机等设备；

以上所有的设备都通过总线连接，总线相当于人的神经

现代计算机的结构更复杂，包括 **多重总线**

上课开始，老师讲课，学生听课，老师是程序员，学生是计算机，学生的器官都是计算机各部分组成

- 1、你通过耳朵接收老师讲的知识（输入）
- 2、通过自己的神经，将接收的数据存入自己的内存/短期记忆（总线、内存）
- 3、光听不行，你还需要反应（处理）老师讲的知识，于是你的大脑（cpu）从短期记忆里取出知识（指令），分析知识（指令），然后学习知识/执行指令（cpu取指、分析、执行）
- 4、你通过作业或者说话输出你学到的结果
- 5、你想要永久将知识保存下来，只能拿出一个笔记本，把刚刚学会的知识都写到本子上，这个本子就是硬盘（磁盘）

综合上面所说的，我们知道其实计算机是由五大单元：**CPU（控制单元、算术逻辑单元）、主存储器、输入单元、输出单元**构成的。也可以说CPU+主存储器+输入输出构成了电子计算机的三大核心组件：

- 1、控制单元+算术逻辑单元=>CPU
- 2、主存储器，即内存
- 3、输入单元Input+输出单元Output=>IO设备

1.2 进制

进制也就是进位计数制，是人为定义的带进位的计数方法。对于任何一种进制—X进制，就表示每一位上的数运算时都是逢X进一位。十进制是逢十进一，十六进制是逢十六进一，二进制就是逢二进一，以此类推，x进制就是逢x进位。

1.2.1 十进制

由于人类解剖学的特点，双手共有十根手指，故在人类自发采用的进位制中，十进制是使用最为普遍的一种。成语“屈指可数”某种意义上来说描述了一个简单计数的场景，而原始人类在需要计数的时候，首先想到的就是利用天然的算筹——手指来进行计数。

数值本身是一个数学上的抽象概念。经过长期的演化、融合、选择、淘汰，系统简便、功能全面的十进制计数法成为人类文化中主流的计数方法，经过基础教育的训练，大多数的人从小就掌握了十进制计数方法。盘中放了十个苹果，通过数苹果我们抽象出来“十”这一数值，它在我们的脑海中就以“10”这一十进制编码的形式存放和显示，而不是其它的形式。从这一角度来说，十进制编码几乎就是数值本身。

十进制的基数为10，数码由0-9组成，计数规律**逢十进一**。

十进制数：

```
1 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23...
```

1.2.2 二进制

二进制有两个特点：它由两个数码0，1组成，二进制数运算规律是**逢二进一**。

为区别于其它进制，二进制数的书写通常在数的右下方注上基数2，或在后面加B表示，其中B是英文二进制 **Binary** 的首字母。

例如：二进制数10110011可以写成 $(10110011)_2$ ，或写成 **10110011B**。对于十进制数可以不加标注，或加后缀D，其中D是英文十进制Decimal的首字母D。

- 二进制数：

```
1 0,1,10,11,100,101,110,111,1000,1001,1011,1100,1101,1110,1111,10000,10001,10010,10011,10100,10101,10110...
```

1.2.3 八进制

由于二进制数据的基数R较小，所以二进制数据的书写和阅读不方便，为此，在小型机中引入了八进制。八进制的基数 $R=8=2^3$ ，有数码0、1、2、3、4、5、6、7，并且每个数码正好对应三位二进制数，所以八进制能很好地反映二进制。八进制用下标8或数据后面加O（Octal）表示 例如：二进制数据 $(11101010.010110100)_2$ 对应八进制数据 $(352.264)_8$ 或 **352.264O**。

- 八进制数：

```
1 0,1,2,3,4,5,6,7,10,11,12,13,14,15,16,17,20,21,22,23,24,25,26,27,30,31,32,33...
```

1.2.4 十六进制

由于二进制数在使用中位数太长，不容易记忆，所以又提出了十六进制数。十六进制数有两个基本特点：它由十六个数码：数字0~9加上字母A-F组成（它们分别表示十进制数10~15），十六进制数运算规律是逢十六进一，即基数 $R=16=2^4$ ，通常在表示时用尾部标志H或下标16以示区别，在c语言中用添加前缀0x以表示十六进制数。

例如：十六进制数 **4AC8** 可写成 $(4AC8)_{16}$ ，或写成 **4AC8H**、**0x4AC8**。

- 十六进制数：

```
1 0,1,2,3,4,5,6,7,8,9,a,b,c,d,e,f,10,11,12,13,14,15,16,17,18,19,1a,1b,1c,1d,1e,1f,20,21,22,23...
```

1.3 进制转换

1.3.1 位权

一个十进制数110，其中百位上的1表示1个 10^2 ，即100，十位的1表示1个 10^1 ，即10，个位的0表示0个 10^0 ，即0。

一个二进制数110，其中高位的1表示1个 2^2 ，即4，低位的1表示1个 2^1 ，即2，最低位的0表示0个 2^0 ，即0。

一个八进制数110，其中高位的1表示1个 8^2 ，即64，低位的1表示1个 8^1 ，即8，最低位的0表示0个 8^0 ，即0。

一个十六进制数110，其中高位的1表示1个 16^2 ，即256，低位的1表示1个 16^1 ，即16，最低位的0表示0个 16^0 ，即0。

可见，在数制中，各位数字所表示值的大小不仅与该数字本身的大小有关，还与该数字所在的位置有关，我们称这关系为数的**位权**。

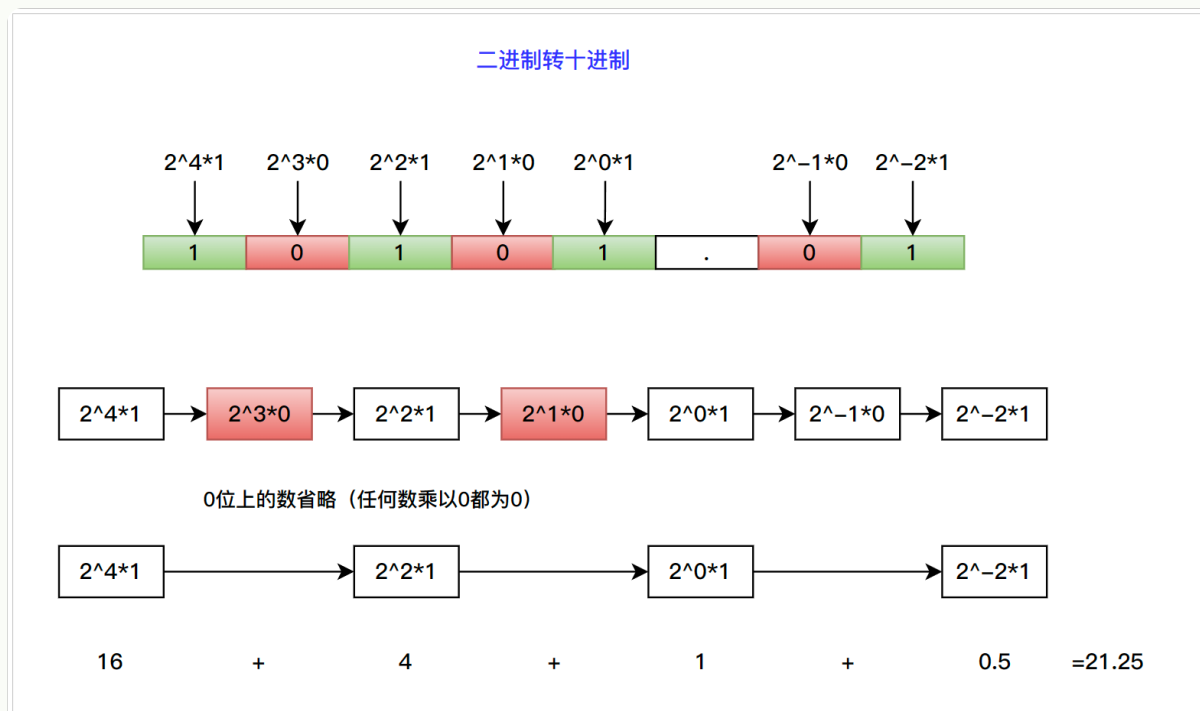
十进制数的位权是以10为底的幂，二进制数的位权是以2为底的幂，八进制数的位权是以8为底的幂，十六进制数的位权是以16为底的幂。数位由高向低，以降幂的方式排列。

1.3.2 按权求和计算

二进制、八进制、十六进制数转换为十进制数的规律是相同的。把x进制数按位权形式展开多项式和的形式，求其最后的和，就是其对应的十进制数——简称“**按权求和**”也叫按权展开计算法；

- 2进制转10进制：

$(10101.01)_2$ 转 10 进制：



最终求得结果： $(10101.01)_2 = (21.25)_{10}$

- 8进制转10进制：

$(306)_8$ 转 10 进制

八进制转十进制

$$\begin{array}{ccc} 8^2 \times 3 & 8^1 \times 0 & 8^0 \times 6 \\ \downarrow & \downarrow & \downarrow \\ \boxed{3} & \boxed{0} & \boxed{6} \\ 192 & + & 6 = 198 \end{array}$$

最终求得结果: $(306)_8 = (198)_{10}$

- 16进制转10进制:

$(1a8)_{16}$ 转 10进制

十六进制转十进制

$$\begin{array}{ccc} 16^2 \times 1 & 16^1 \times 10 & 16^0 \times 8 \\ \downarrow & \downarrow & \downarrow \\ \boxed{1} & \boxed{a} & \boxed{8} \\ 256 & + & 160 + 8 = 424 \end{array}$$

最终求得结果: $(1a8)_{16} = (424)_{10}$

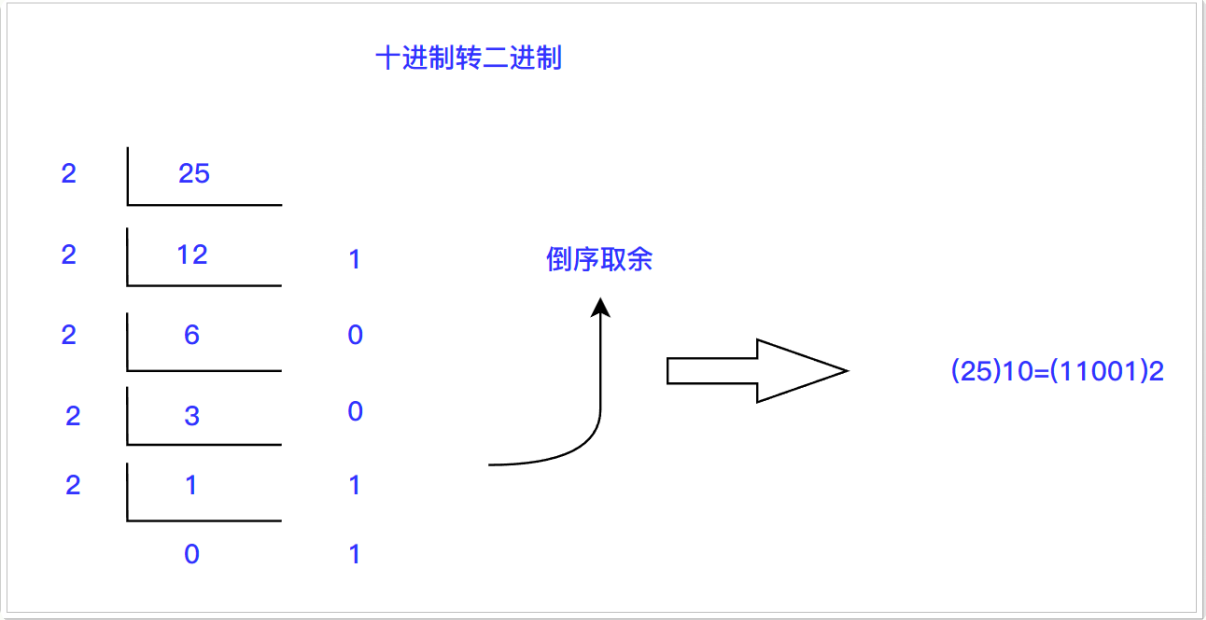
1.3.3 倒数取余计算

一个10进制数转换为一个R进制数采用除以R取余法,即用R连续除以10进制,直到商为0,最终按照倒序的方法得到余数即可,这种简单的计算方法我们陈伟倒数取余法;

R表示进制，10进制转换为2进制则一直除以2得到余数，然后倒数取余，10进制转换为8进制则一直除以8得到余数之后倒数取余，以此类推；

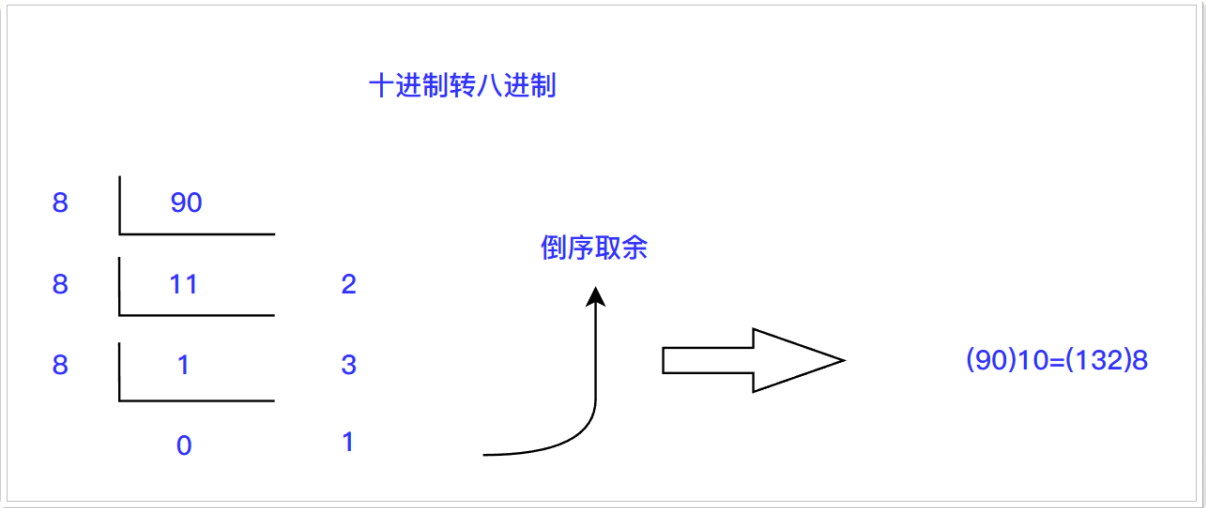
- 10进制转2进制：

25D转二进制



- 10进制转8进制：

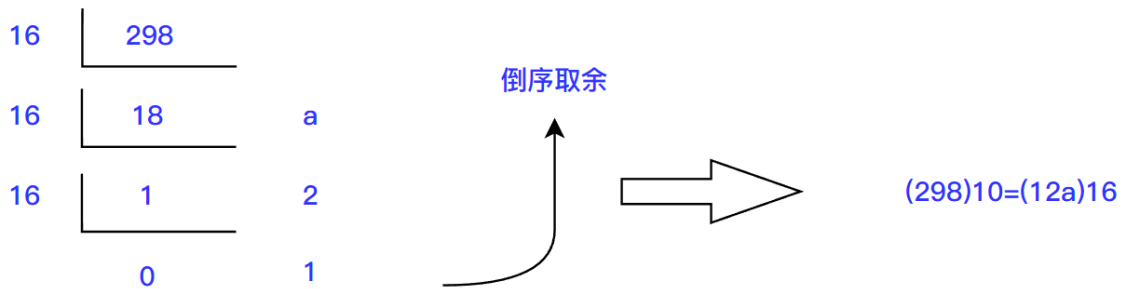
90D转八进制



- 10进制转16进制：

298D转十六进制

十进制转十六进制

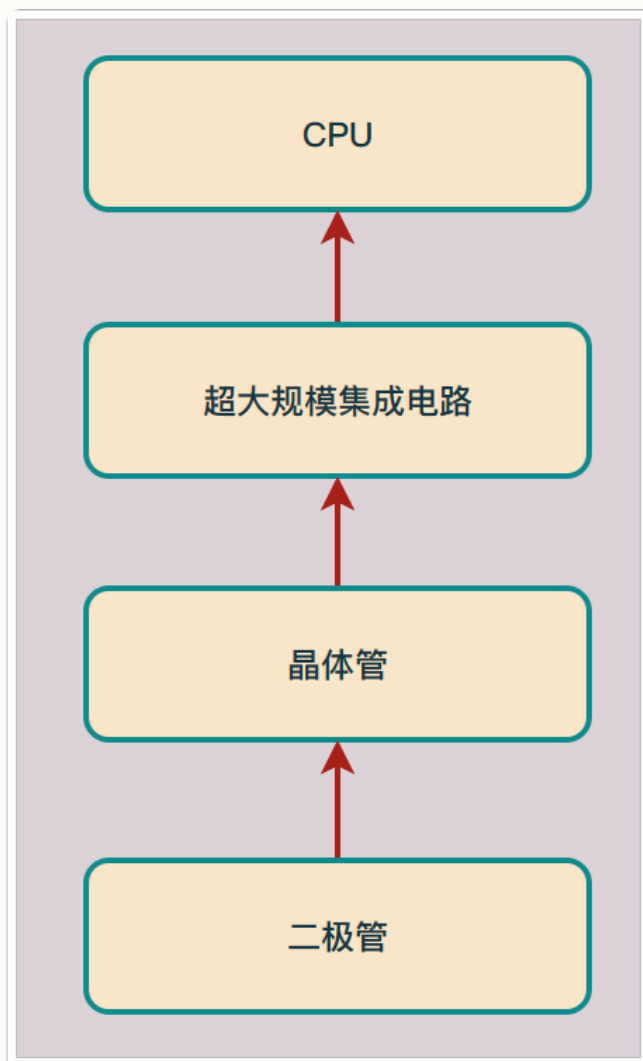


1.4 计算机底层的运算

1.4.1 为什么采用二进制

我们都知道计算机底层采用的是二进制码，即计算机底层存储的全都是0和1，不管是我们看到的视频、图片、音乐、文档和其他任何存储在电脑上的文件，其底层都是0，1，那么为什么要采用0和1来进行存储呢？0和1又是如何变成我们需要的文件呢？

我们知道计算机底层都是靠CPU来进行运算的，CPU的发展经历了如下几个过程：



也就是说CPU最原先是通过二极管发展而来，而二极管只有两个状态：通（1）、断（0）

那么1个二极管能够表示几种状态呢？很明显就两种：0和1

2个二极管可表示4个状态：

```
1 00,01,10,11
```

3个二极管可表示8个状态：

```
1 000,001,010,011,100,101,110,111
```

4个二极管就可表示16个状态，5个表示32个状态，6表示64个状态...而目前CPU都集成了超大规模的集成电路，多大呢？早在2012年Intel发布的Core i7处理器的芯片集成度达到了14亿个晶体管，数据计算能力可想而知；

那么计算机中存储的二进制是如何转变为我们看到的内容呢？

这中间就存在一个编码与解码的过程，其实我们在计算机中打开任何的一个文件、应用程序、游戏等其实都可以看做是一个解码的过程，只不过对应的解析器不同罢了；

以最普通的文本为例，我们跟计算机双方约定好当在键盘上输入一个字母A时，保存到电脑上的内容就为0100 0001，当我们在电脑上输入一个字母B时，电脑底层存储的就是0100 0010，越来越多的信息需要通过编码存储到计算机内，因此编码表这个概念就出来了，世界上的一些权威组织就开始着手于编码的制作，有了编码表我们就能够使用0和1来代表世间上的万物了；编码表就相当于一个翻译官，充当人类与计算机之间的一个翻译官

目前市面上有非常多的编码表，每个表码表的编码内容是不一样的，因此我们存储到计算机时采用的编码表应该与我们取出数据解码时的编码表一致；

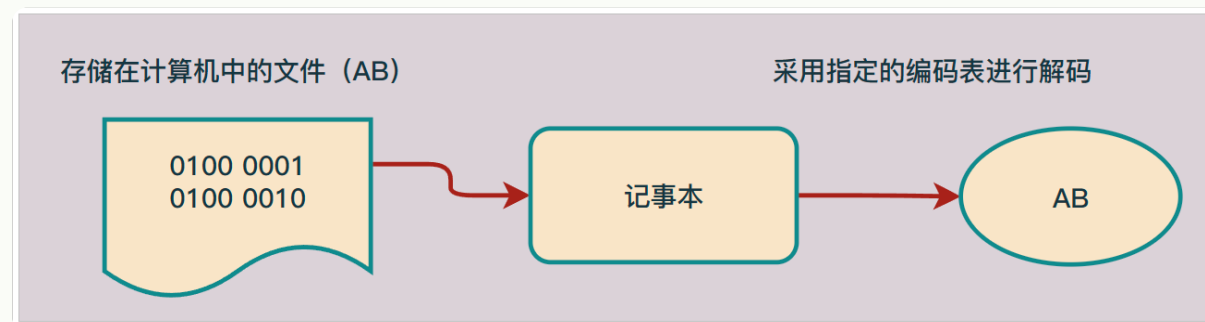
下图是ASCII码表中规定的内容：

```
1 0100 0001 A
2 0100 0010 B
3 0100 0011 C
4 0100 0100 D
5 0100 0101 E
6 0100 0110 F
7 0100 0111 G
8 0100 1000 H
9 0100 1001 I
10 0100 1010 J
11 0100 1011 K
```

ASCII码表：<http://ascii.911cha.com/>

UTF8码表：http://www.mytju.com/classcode/tools/encode_utf8.asp

GBK码表：<https://www.toolhelper.cn/Encoding/GBK>



1.4.2 原码、反码、补码

计算机为了区分数值的正负提出了符号位的设定，计算机用最高位存放符号，这个被称为符号位。正数的符号位为0，负数的符号位为1。

例如，1的二进制表示形式为 0000 0001，而-1的二进制表示形式为 1000 0001。另外在计算机中，一个字节为8个位，最大值为 0111 1111(127)，最小值为 1000 0000(-128)，取值范围为-128~127之间。

1) 原码

原码就是十进制数在二进制中的体现，该二进制包含符号位。

Tips: 原码就是我们之前使用的二进制的格式，只不过我们之前一直没有使用二进制描述过负数，使用原码描述负数只要将符号位设定为1即可。

但仅仅是存在原码对于计算机进行运算时有着巨大缺陷；**那就是对于负数的计算。**

- 对于正数的计算，原码不会有问题：计算2+2

```
1  0000 0010
2 +0000 0010
3 -----
4  0000 0100
```

(0000 0100)的十进制为4，计算正确。

- 对于负数的运算，发现结果与目标值不对：计算2+(-2)

```
1  0000 0010
2 +1000 0010
3 -----
4  1000 0100
```

(1000 0100)的十进制为-4，计算错误。

为了解决原码不能用于计算负数的这种问题，因此迫切需要一种新的表示方法来解决这个巨大的缺陷，于是反码出现了。

2) 反码

反码的存在是为了正确计算负数，因为原码不能用于计算负数。

- 正数的反码：反码就等于原码。**
- 负数的反码：原码的符号位不变，其余位取反。**

十进制数字	原码	反码
1	0000 0001	0000 0001
-1	1000 0001	1111 1110
2	0000 0010	0000 0010
-2	1000 0010	1111 1101

对于通常的负数计算，使用反码可以完成，例如我们计算-4-1：

-4的原码：1000 0100，反码为：1111 1011

```
1  1111 1011 (-4)
2 -0000 0001 (1)
3 -----
4  1111 1010
```

计算结果为：1111 1010（反码），转换为原码为：1000 0101，结果为-5，刚好是正确的；

但反码也有它的缺陷，在计算结果为0时会出现问题。

- 计算结果为0时：

这个时候再来计算2+(-2)：

-2的原码为：1000 0010，反码为：1111 1101

```
1  0000 0010 (2)
2 +1111 1101 (-2)
3 -----
4  1111 1111
```

然后将计算的结果(1111 1111)转回原码(1000 0000)，结果为-0。那么问题来了，-0是0吗？在二进制中出现了两个可以描述“0”这个数值的二进制，一个为(1000 0000)，另一个为(0000 0000)，这很显然有问题；

3) 补码

为了解决上面的问题，补码出现了；

正数的补码是其本身，负数的补码等于其反码 +1

十进制数字	原码	反码	补码
1	0000 0001	0000 0001	0000 0001
-1	1000 0001	1111 1110	1111 1111
2	0000 0010	0000 0010	0000 0010
-2	1000 0010	1111 1101	1111 1110

- 使用补码计算结果为0的情况：

这个时候再来计算2+(-2)：

-2的原码：1000 0010，反码：1111 1101，补码：1111 1110

2的原码、反码、补码都为本身，即：0000 0010

```
1  1111 1110 (-2)
2 +0000 0010 (2)
3 -----
4  0000 0000 (0)
```

结果为0；

在计算机当中都是使用补码来进行计算和存储的。补码很好的解决了反码相加结果为0的问题，并且补码还可以记录一个特殊的值 -128，这个数据在1个字节下是没有原码和反码的。

思考一个问题：127+1=？

127的补码：0111 1111

1的补码: 0000 0001

```
1 0111 1111 (127)
2 +0000 0001 (1)
3 -----
4 1000 0000 (-128)
```

高位为1, 说明是负数, 其结果为"-0", 另外需要注意的是, -0的补码用来表示-128, -128并没有原码和反码的表示方法;

在八位二进制下, 因为有一位是符号位, 原码和反码只能表示0到127, -0到-127, 所以-128不能用原码或反码表示, 但计算机科学家们并没有把-0这个数浪费, 而是让其表示-128。

不仅 $127+1=-128$, 另外 $-128-1$ 的结果也会发生改变;

-128的补码: 1000 0000

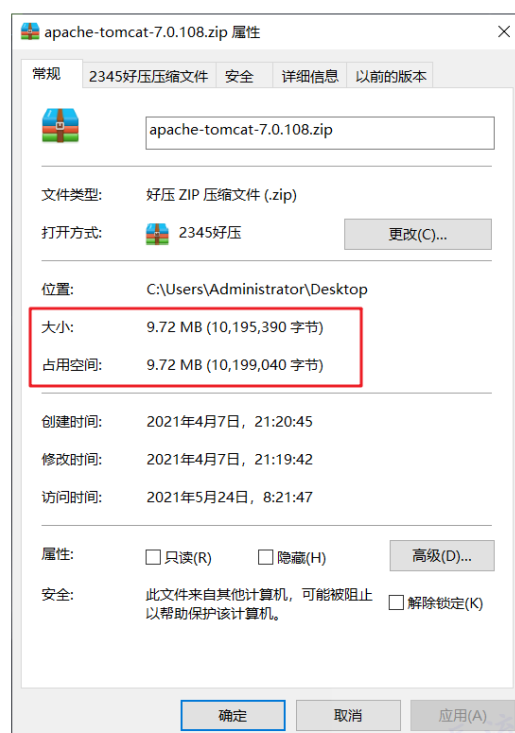
-1的补码: 1000 0001, 反码: 1111 1110, 补码: 1111 1111

```
1 1000 0000 (-128)
2 +1111 1111 (-1)
3 -----
4 0111 1111 (127)
```

最终计算的结果为127

1.5 存储单位

计算机的最小存储单位是**字节**, 计算机存储任何的数据, 都是以字节的形式存储, 右键点击文件属性, 我们可以查看文件的字节大小



1.5.1 基本的存储单位

- 位（bit）：二进制数中的一个数位，可以是0或者1，是计算机中数据的最小单位。
- 字节（Byte，B）：计算机中数据的基本单位，每8位组成一个字节。各种信息在计算机中存储、处理至少需要一个字节。例如，一个ASCII码用一个字节表示，一个汉字用两个字节表示。

1.5.2 其他存储单位

在计算机各种存储介质（例如内存、硬盘、光盘等）的存储容量表示中，用户所接触到的存储单位不是位、字节和字，而是KB、MB、GB等，但这不是新的存储单位，而是基于字节换算的。

1B=8bit

KB：1KB=1024B 。早期用的软盘有360KB和720KB的，不过软盘已经很少使用。

MB：1MB=1024KB。早期微型机的内存有128MB、256MB、512MB，目前内存都是1GB、2GB甚至更大。

GB：1GB=1024MB。早期微型机的硬盘有60GB、80GB，目前都是500GB、1TB甚至更大。

TB：1TB=1024GB。目前个人用的微型机存储容量也都能达到这个级别了，而作为服务器或者专门的计算机，不可缺少这么大的存储容量。

换算率约等于1000（1024），从大到小顺序为T、GB、MB、KB、B再小就是位了。

计算机存储单位一般用bit、B、KB、MB、GB、TB、PB、EB、ZB、YB、BB、NB、DB……来表示，它们之间的关系是：

转换率	单位
一个二进制，0或1	bit
1B=8bit	B
1MB=1024B	MB
1GB=1024MB	GB
1TB=1024GB	TB
1PB=1024TB	PB
1EB=1024PB	EB
1ZB=1024EB	ZB
1YB=1024ZB	YB
1BB=1024YB	BB
1NB=1024BB	NB
1DB=1024NB	DB

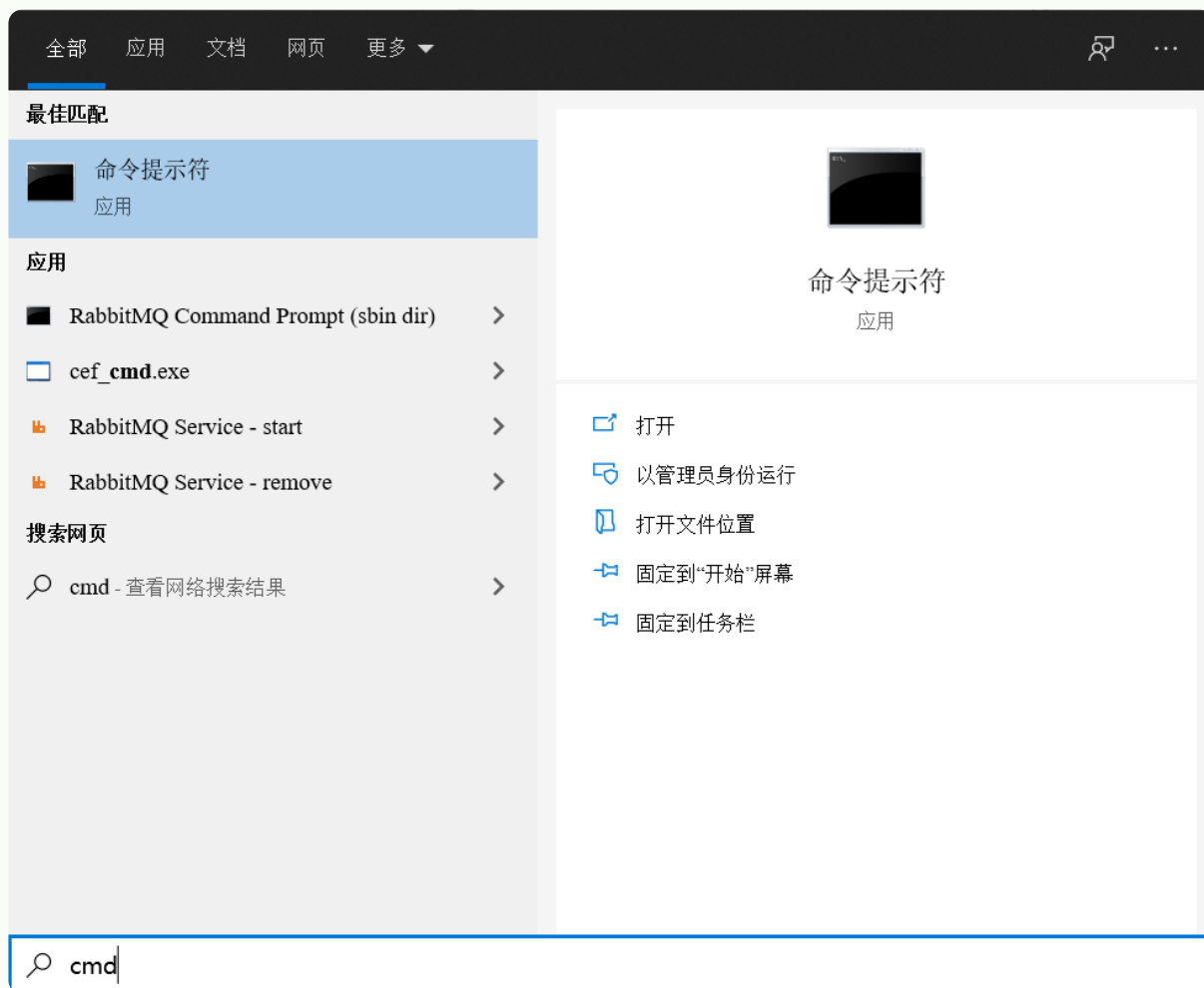
需要注意的是：计算机最小的存储单位虽然是位（bit），但是计算机的基本存储单位是字节，又名Byte，用大写字母B表示

1.6 常用DOS命令

DOS是Disk Operating System的缩写，即磁盘操作系统。它是一个基于磁盘管理的操作系统，在微软公司的Windows2000出版之前，DOS 系统基本统治着个人操作系统世界。即便是Windows3.x/9x都是建立在DOS平台之上的大型GUI界面应用程序。随着Windows操作系统的风行，DOS系统已逐渐成为一种历史，失去了往日那样的熠熠生辉。但是别看Windows风光无限，但是还是有很多的很难解决或者无法解决的的问题，而这个时候我们的DOS系统就可以大显身手了，用DOS命令来解决一些问题，往往会收到事半功倍的效果。

- 进入DOS命令窗口：

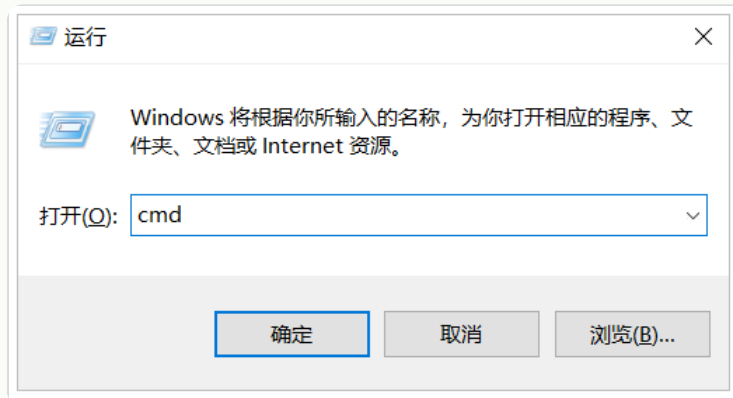
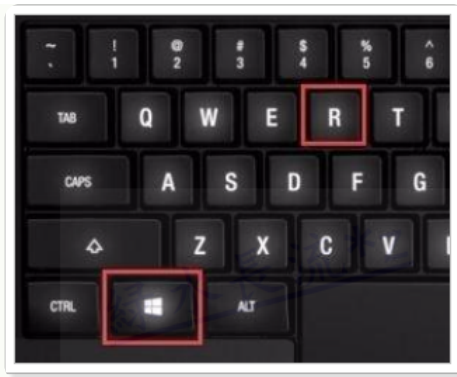
按一下windows键，直接输入cmd：



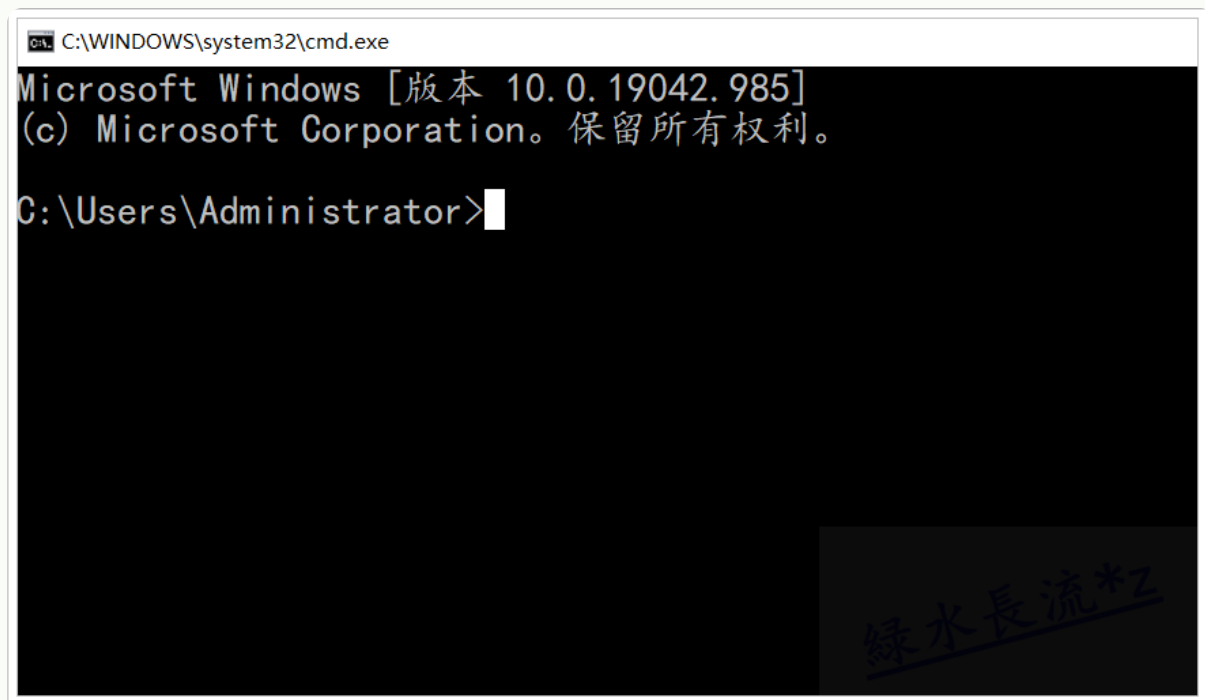
windows键：



或者按 **windows+R** 键：



cmd窗口:



Dos常用命令:

命令	解释
盘符名:	盘符切换
dir	查看当前文件夹内容
cd 文件夹名	进入指定文件夹
cd..	返回上一级文件夹

命令	解释
cd\	退出到磁盘根目录
cls	清屏

二、Java概述

2.1 编程语言

程序设计语言是人们为了描述解题步骤（即编程序）而设计的一种具有语法语义描述的记号。计算机语言的种类非常的多，总的来说可以分成**机器语言**、**汇编语言**、**高级语言**三大类。

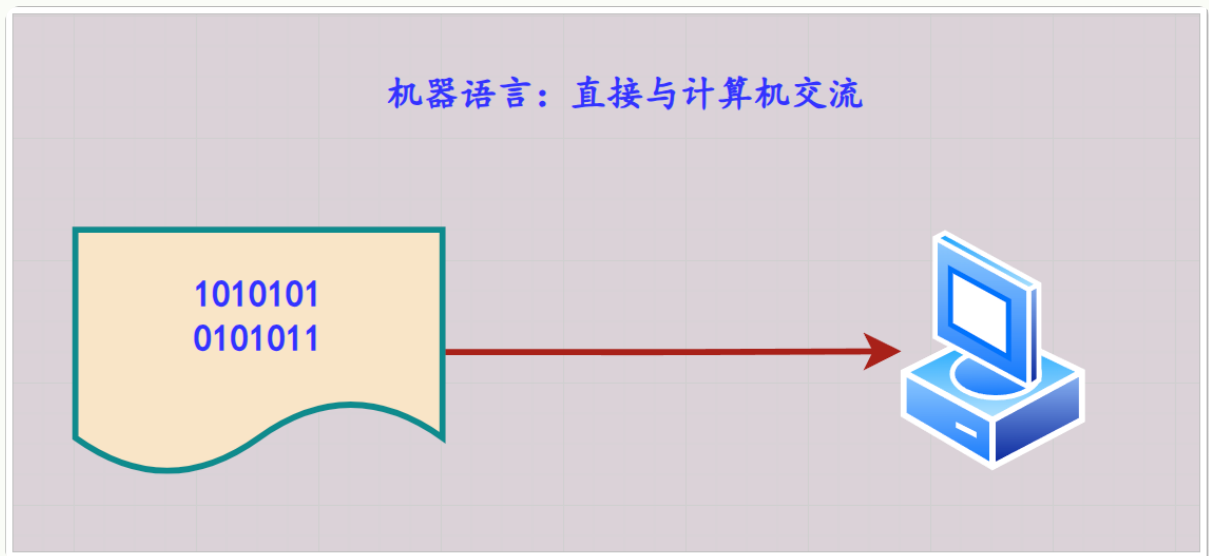
在计算机角度，每一种CPU类型都有自己可以识别的一套指令集，计算机不管你这个程序是用什么语言来编写的，其最终只认其CPU能够识别的二进制指令集。

2.1.1 机器语言：

在早期计算机刚发展的时代，人们都是直接输入01010101这样的没有语义的二进制指令（俗称机器语言）来让计算机工作的，可读性几乎没有，没人愿意直接编写那些没有可读性、繁琐、费时，易出差错的二进制01代码，所以后来才出现了编程语言（包括汇编与高级语言）。

Tips: 编程语言的诞生，使得人们编写的代码有了可读性，有了语义，与直接用01相比，更有利于记忆。

- 机器语言执行过程：



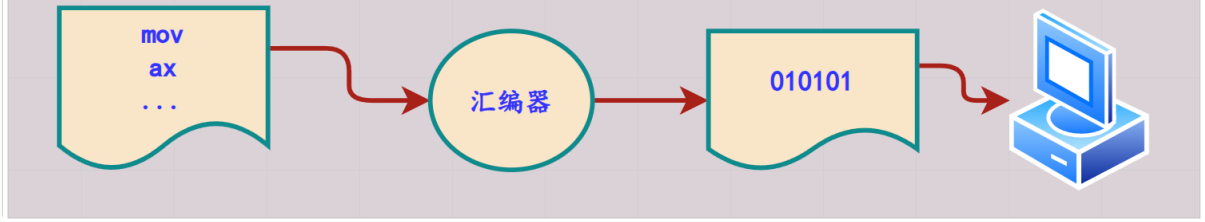
机器语言的特点是：代码编写复杂、易错、没有语义，对程序员非常不友好，但由于是机器指令，可以直接被计算机所识别，**效率高**；

2.1.2 汇编语言：

为了克服面向CPU的指令集的难读、难编、难记和易出错的缺点，后来就出现了面向特定CPU的特定**汇编语言**，比如我打上这样的x86汇编指令 **mov**、**ax**，然后用上机器码做的汇编器，它将会被翻译成 1000100111011000 这样的二进制01格式的机器指令；

- 汇编语言执行过程：

汇编语言：借助汇编器进行翻译



汇编语言的特点是：相对于机器指令而言，代码编写较为简单，程序被赋予了语义，对程序员友好，代码需要借助汇编器转换为机器指令，效率比机器指令略低；

汇编的实质是机器指令（机器码）的助记符，是一种低级符号语言（语义不够强），机器指令集是一款CPU的编程特征，是这款CPU的设计者制定的，CPU的内部电路设计就是为了实现这些指令集的功能。

CPU常见的架构有：arm架构，x86架构，mips架构等；汇编语言是针对某一个CPU而写的，不能编译到另一个CPU。

例如：`add eax,ebx` 就只是X86 汇编的。而mips的，可能是 `add $t0,$t0,$t1`

不同CPU架构上的汇编语言指令不同，不同操作系统之间的指令也可能不一样，而为了统一套写法，同时又不失汇编的表达能力，因此高级编程语言就诞生了；

2.1.3 高级语言：

用高级语言写的代码，会被编译器先转换成对应平台的 **汇编指令**，再转成机器码，最后将这些过程中产生的中间模块链接成一个可以被操作系统执行的程序。

高级语言代码

汇编编译器

汇编指令

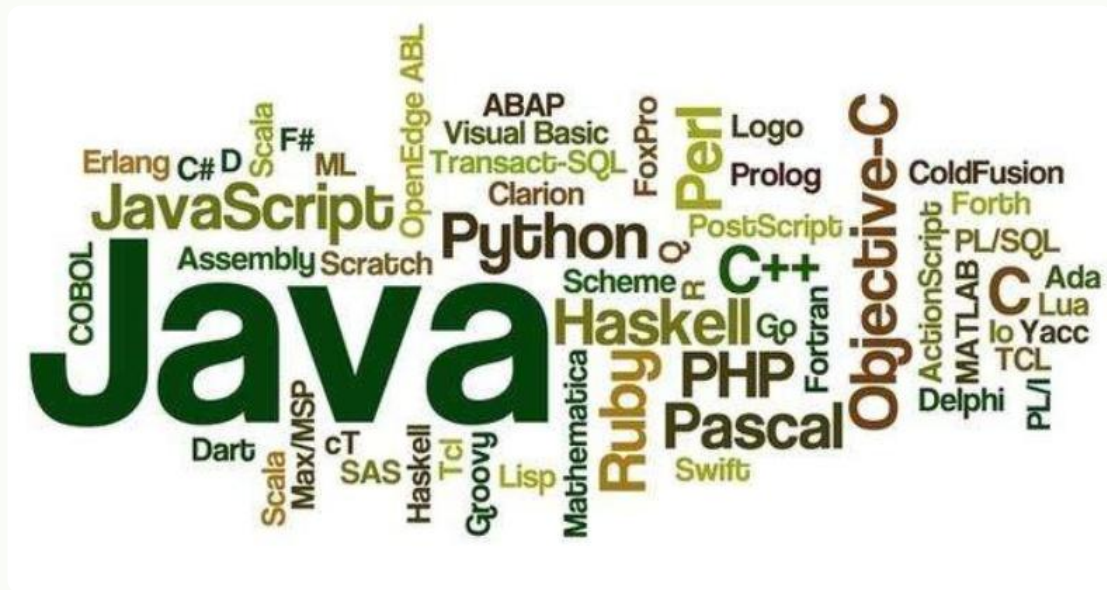
汇编器

机器指令



那么汇编语言和高级语言比较，我们就不需要去阅读特定CPU的汇编码，我只需要写通用的高级语言的源代码就可以实现程序的编写，我们用将更偏机器实现的汇编语言称为低级语言，与汇编相比之上的语言，我们称之为高级语言；

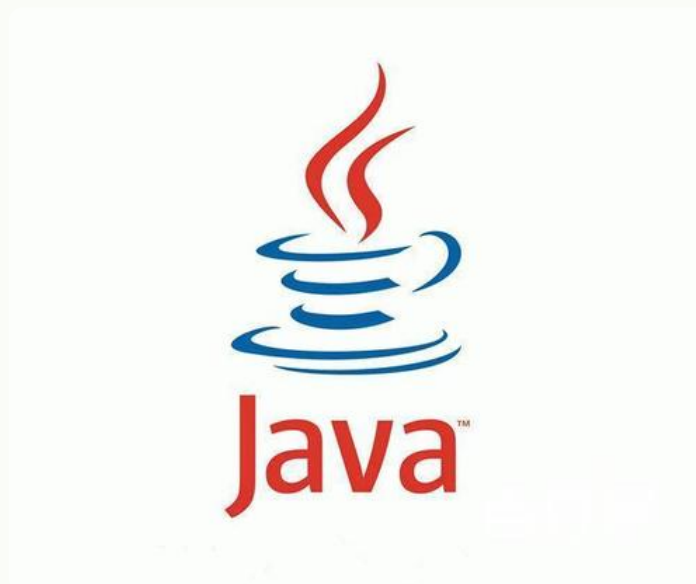
我们常见的一些编程语言都是高级语言，都处于汇编语言之上：



2.2 Java编程语言介绍

2.2.1 什么是Java

Java是1995年美国的Sun公司（Stanford University Network/斯坦福大学网络公司）推出的一门面向对象的 **高级编程语言**



其他高级编程语言推出时间：

C: 1972 年

C++: 1983年

Python: 1991年

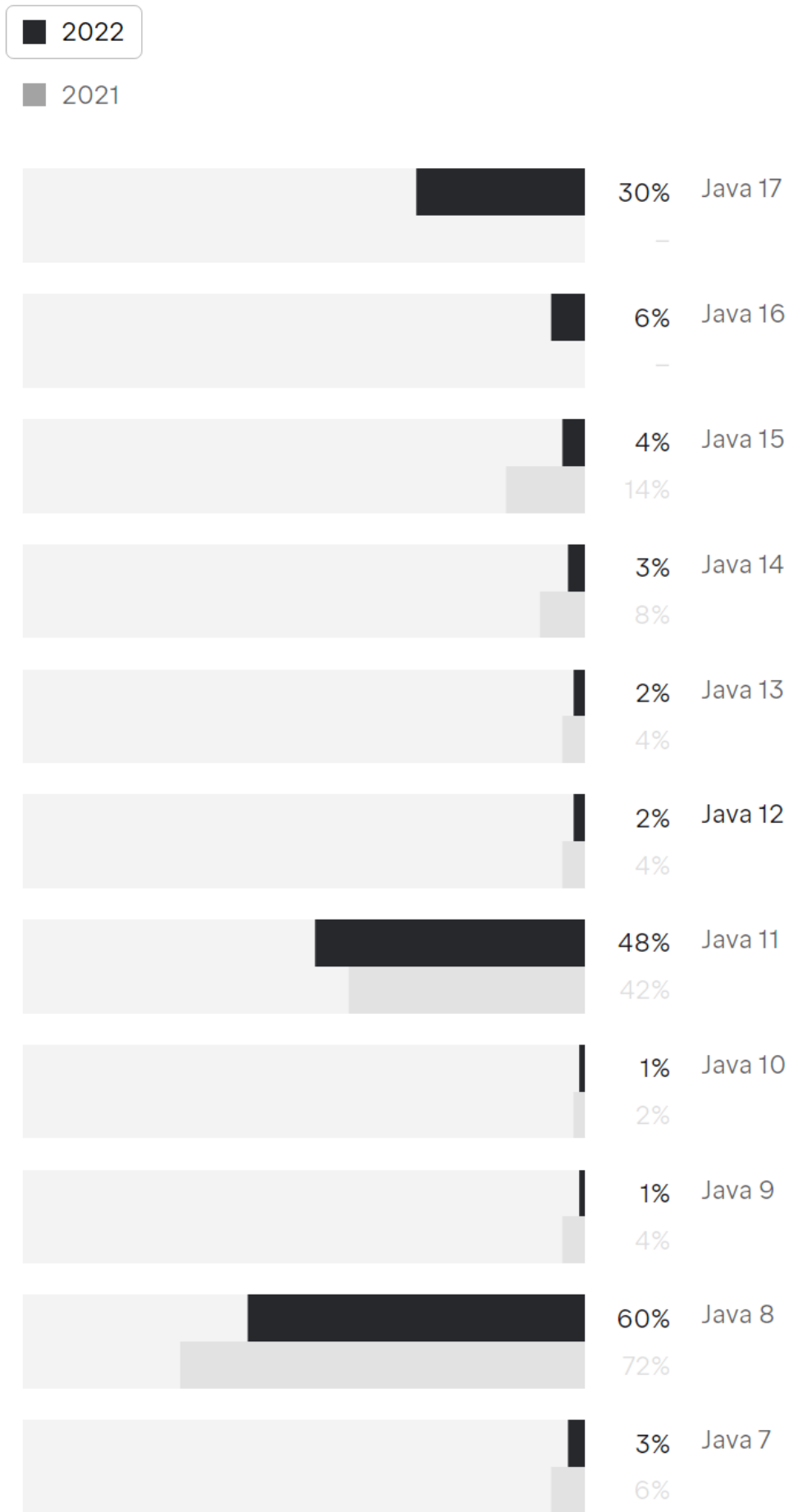
2.2.2 Java的发展历史



詹姆斯·高斯林（James Gosling）1977年获得了加拿大卡尔加里大学计算机科学学士学位，1983年获得了美国卡内基梅隆大学计算机科学博士学位，毕业后到IBM工作，设计IBM第一代工作站NeWS系统，但不受重视。后来转至Sun公司，1990年，与Patrick, Naughton和Mike Sheridan等人合作“绿色计划”，后来发展一套语言叫做“Oak”，后改名为Java。Java是印度尼西亚 爪哇岛 的英文名称，爪哇岛因盛产咖啡而闻名。Java创始人员团队中其中一名成员想起自己在Java岛（爪哇岛）上曾喝过一种美味的咖啡。

- 1995年Sun公司发布Java1.0版本
- 1997年发布Java 1.1版本
- 1998年发布Java 1.2版本
- 2000年发布Java 1.3版本
- 2002年发布Java 1.4版本
- **2004年发布Java 1.5版本**
- 2006年发布Java 1.6版本
- 2009年Oracle甲骨文公司收购Sun公司，并于2011发布Java 1.7版本
- **2014年发布Java 1.8版本**
- 2017年发布Java 9.0版本
- 2018年3月发布Java10.0版本
- **2018年9月发布Java11版本**
- 2019年3月发布Java12版本
- 2019年9月发布Java13版本
- 2020年7月发布Java14版本
- 2021年1月发布Java15版本
- 2021年4月发布Java16版本
- **2021年9月发布Java17版本**
- 2022年3月发布Java18版本
- 2022年9月发布Java19版本
- 2023年3月发布Java20版本
- 2023年9月发布Java21版本

下图是Jetbrains idea IDE 对Java各版本的使用统计情况（2021年~2022年）：



多年来，Java 8一直是领先的Java版本。然而，它的使用率在2022年下降了12个百分点。这并不奇怪，因为在2022年3月，Java 8失去了Oracle Premier支持。因此，许多开发人员决定将他们的应用程序迁移到不同的版本。Java 17的出现，一个新的长期支持版本，吸引了很多用户。另外，Java 11和Java 16等其他版本也获得了更多的用户。

2.2.3 Java的平台

Java 平台有三个版本, 这使软件开发人员、服务提供商和设备生产商可以针对特定的市场进行开发;

Java SE (Java Platform, Standard Edition): JavaSE以前称为J2SE。它允许开发和部署在桌面、服务器、嵌入式环境和实时环境中使用的Java应用程序。Java SE包含了支持Java Web服务开发的类, 并为Java Platform, Enterprise Edition (Java EE) 提供基础。

Java EE (Java Platform, Enterprise Edition): 这个版本以前称为J2EE。企业版本帮助开发和部署可移植、健壮、可伸缩且安全的服务器端Java应用程序。Java EE是在Java SE的基础上构建的, 它提供Web服务、组件模型、管理和通信API, 可以用来实现企业级的面向服务体系结构 (service-oriented architecture, SOA) 和Web 2.0应用程序。

Java ME (Java Platform, Micro Edition): 这个版本以前称为J2ME。Java ME为在移动设备和嵌入式设备 (比如手机、PDA、电视机顶盒和打印机) 上运行的应用程序提供一个健壮且灵活的环境。Java ME包括灵活的用户界面、健壮的安全模型、许多内置的网络协议以及对可以动态下载的连接和离线应用程序的丰富支持。基于Java ME规范的应用程序只需编写一次, 就可以用于许多设备, 而且可以利用每个设备的本机功能。

三者的区别: 简而言之, 主要是面向的应用领域不一样

- JavaSE: 即J2SE, Java标准版, 主要做一般的Java应用, 比如: 应用软件/QQ之类的通信软件等等.
- JavaEE: 即J2EE, 主要做企业应用, 比如公司网站, 企业解决方案等;
- JavaME: 即J2ME, 主要面向嵌入式等设备应用的开发, 比如手机游戏等

JavaSE是基本, 先学好了这个, 以后, 根据自己的兴趣等, 选JavaME或者JavaEE进行扩展, 就容易多了

2.3 Java能做什么

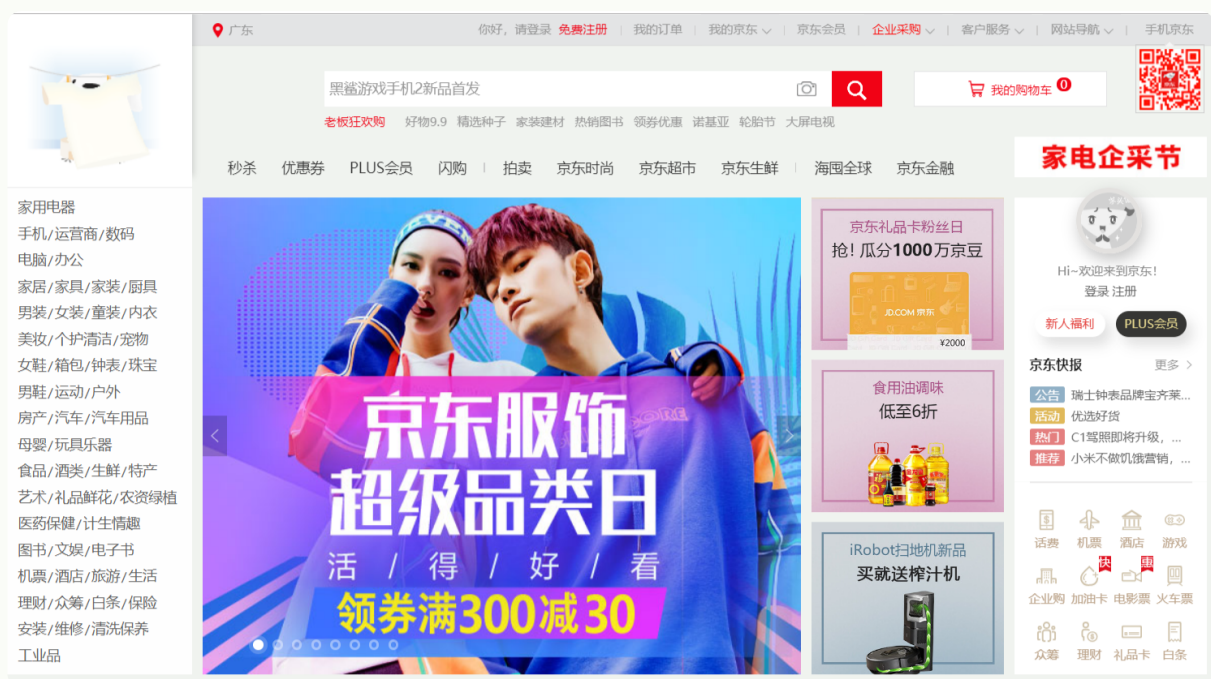
2.3.1 桌面应用程序



2.3.2 嵌入式



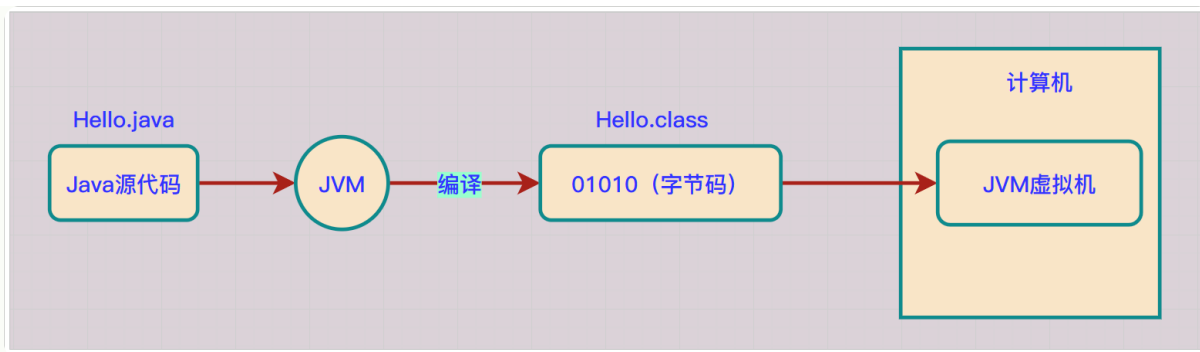
2.3.3 WEB应用程序



三、搭建Java环境

3.1 Java虚拟机

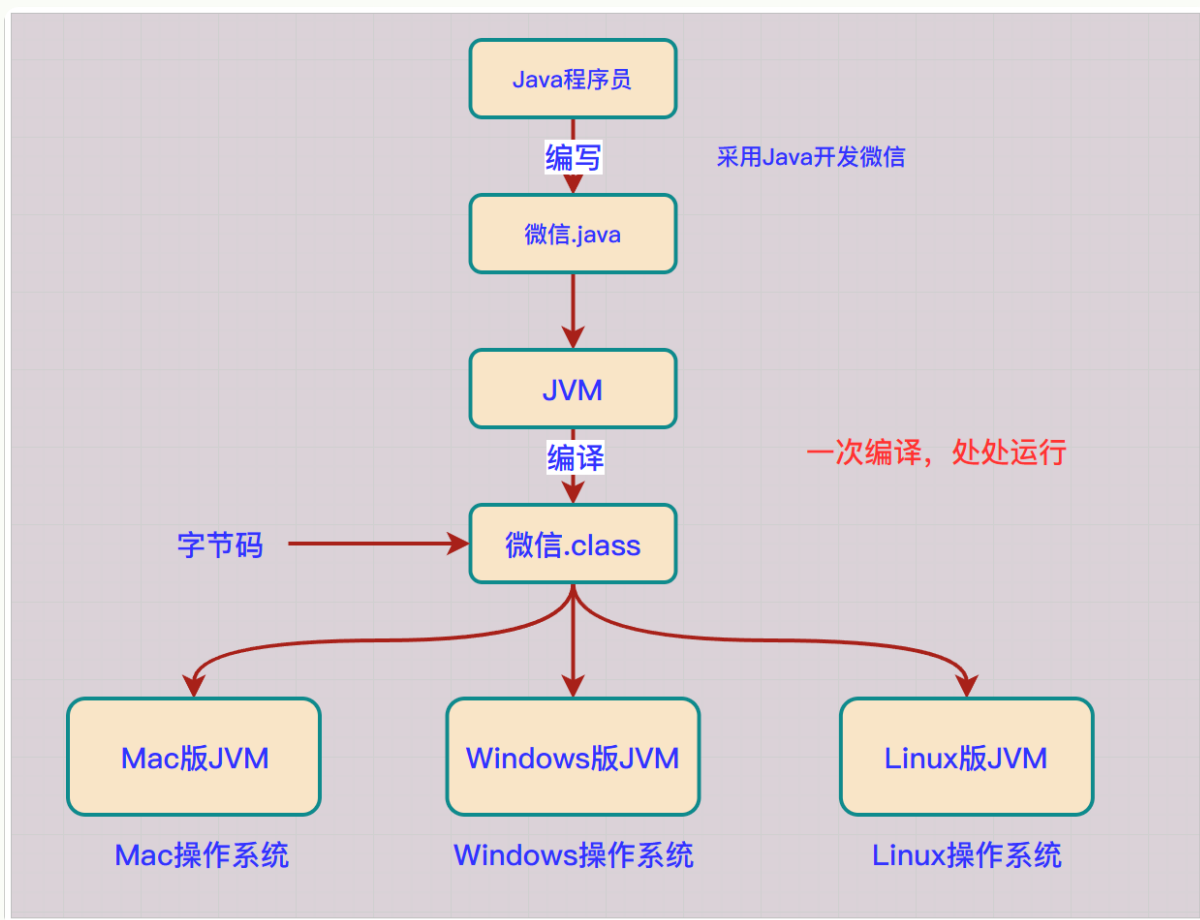
我们编写的所有Java代码最终都是运行在Java虚拟机之上的，如图：



JVM (Java Virtual Machine)：Java虚拟机，简称JVM，是运行所有Java程序的假想计算机，是Java程序的运行环境，是Java 最具吸引力的特性之一。我们编写的Java代码，都运行在 JVM 之上。

3.2 跨平台

跨平台：任何软件的运行，都必须运行在操作系统之上，而我们用Java编写的软件可以运行在任何的操作系统上，这个特性称为Java语言的跨平台特性(“一次编写,处处运行”)。该特性是由JVM实现的，我们编写的程序运行在JVM上，而JVM运行在操作系统上。



Tips: C语言程序与操作系统有关，不同的平台 (Windows、Linux) 对C语言做了不同程度的支持和拓展，而Java语言与操作系统无关，因为Java代码最终运行在JVM上；

3.3 JRE和JDK和JVM的关系

基本语法

- **JDK (Java Development Kit)**：是Java程序开发工具包，包含 JRE 和开发人员使用的工具；
- **JRE (Java Runtime Environment)**：是Java程序的运行时环境，包含 JVM 和运行时所需的**核心类库**；

- **JVM (Java Virtual Machine)** : 是一款Java虚拟机, 模拟Java运行时的一个平台, 对内存分配, 管理、线程调度等都有一定的管理;



总结:

- JDK是一个Java程序员的开发工具包，除了包含JRE之外，还提供有许多Java开发工具给Java程序员使用，如Javac、JMC、Jstack等
- JRE是一个Java的运行环境，除了包含JVM之外，还提供很多的一些JavaAPI类库，为Java程序提供很多功能，如果没有这些类库那么Java程序员需要完成这些功能就需要手动编写这些逻辑代码；
- JVM是一个Java虚拟机，Java程序（class文件）最终是运行在JVM上的，他更倾向于模拟一台真实的计算机，对内存的分配，回收，共享、线程调度等功能都具备一定的管理能力；在运行Java程序的同时调用JRE提供的一些类库，使得Java程序更加强大、方便；

四、编写Java程序

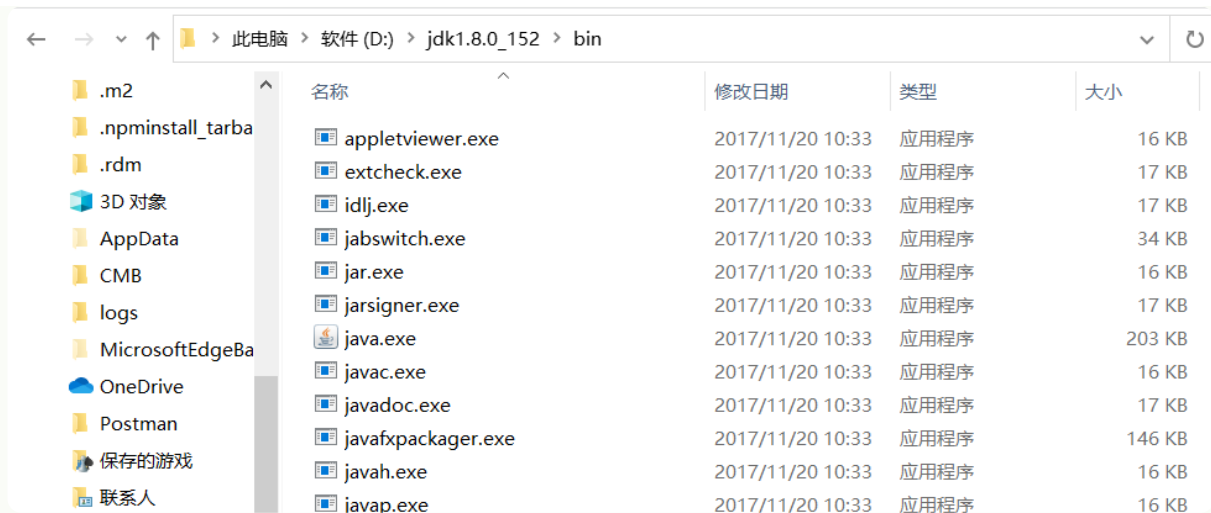
4.1 搭建Java开发环境

4.1.1 准备一个jdk

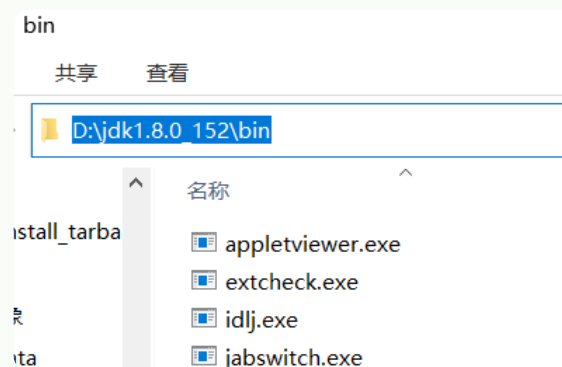
准备一个jdk, 复制到任意一个磁盘或文件夹内:

名称	修改日期	类型	大小
bin	2020/12/25 16:51	文件夹	
db	2020/12/25 16:51	文件夹	
include	2020/12/25 16:51	文件夹	
javafx-src	2020/12/25 16:51	文件夹	
jre	2020/12/25 16:51	文件夹	
lib	2020/12/25 16:51	文件夹	
COPYRIGHT	2017/9/14 3:15	文件	4 KB
javafx-src.zip	2017/11/20 10:33	好压 ZIP 压缩文件	5,081 KB
LICENSE	2017/11/20 10:33	文件	1 KB
README.html	2018/9/25 21:36	HTML 文件	1 KB
release	2017/11/20 10:34	文件	1 KB
src.zip	2017/9/14 3:15	好压 ZIP 压缩文件	20,763 KB
THIRDPARTYLICENSEREADME.txt	2017/11/20 10:33	TXT 文件	142 KB
THIRDPARTYLICENSEREADME-JAVAF...	2017/11/20 10:33	TXT 文件	63 KB

进入 **bin** 目录:



复制当前磁盘目录：`D:\jdk1.8.0_152\bin`



4.1.2 配置环境变量

计算机-->右击-->属性



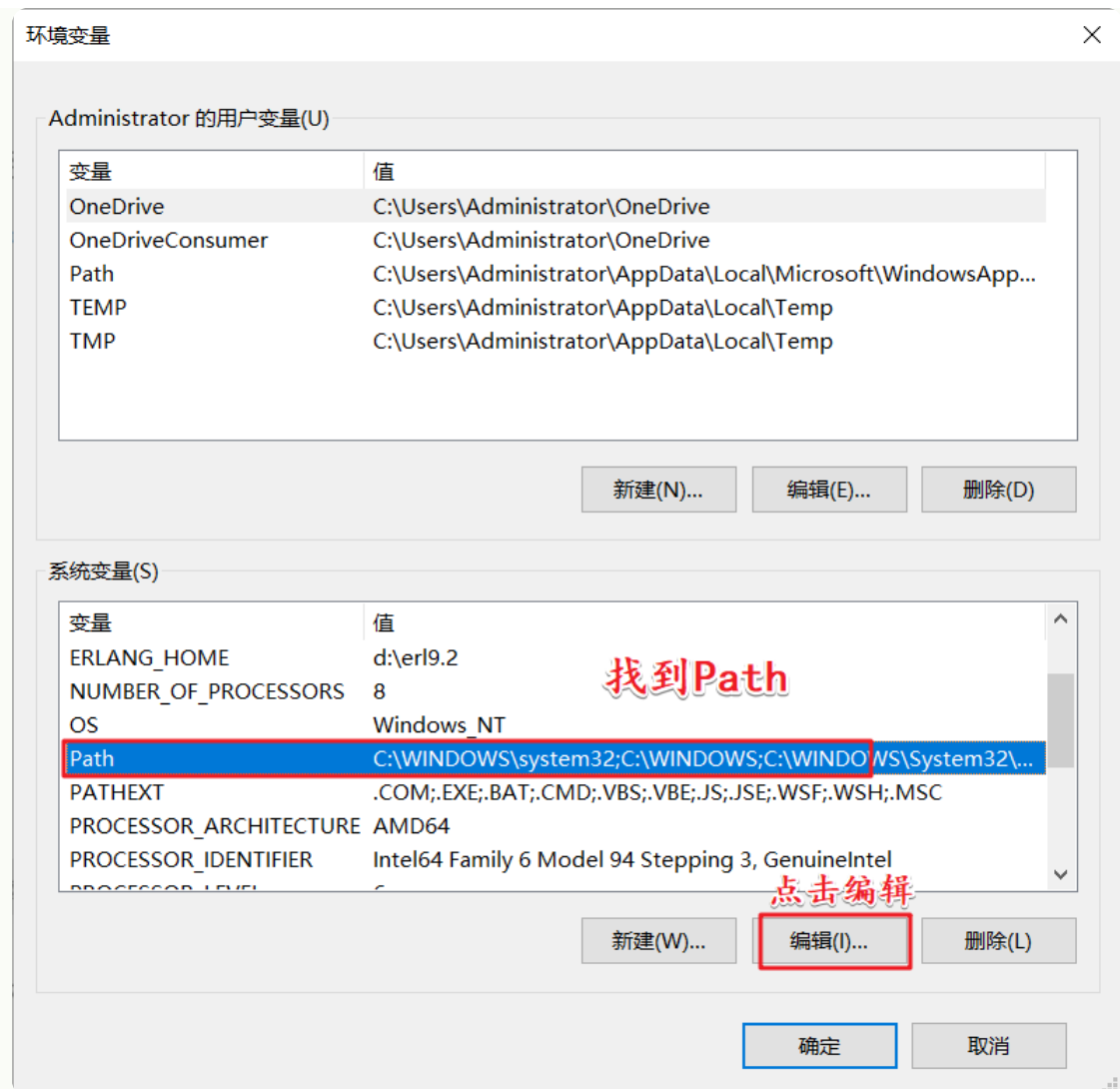
选择高级系统设置：



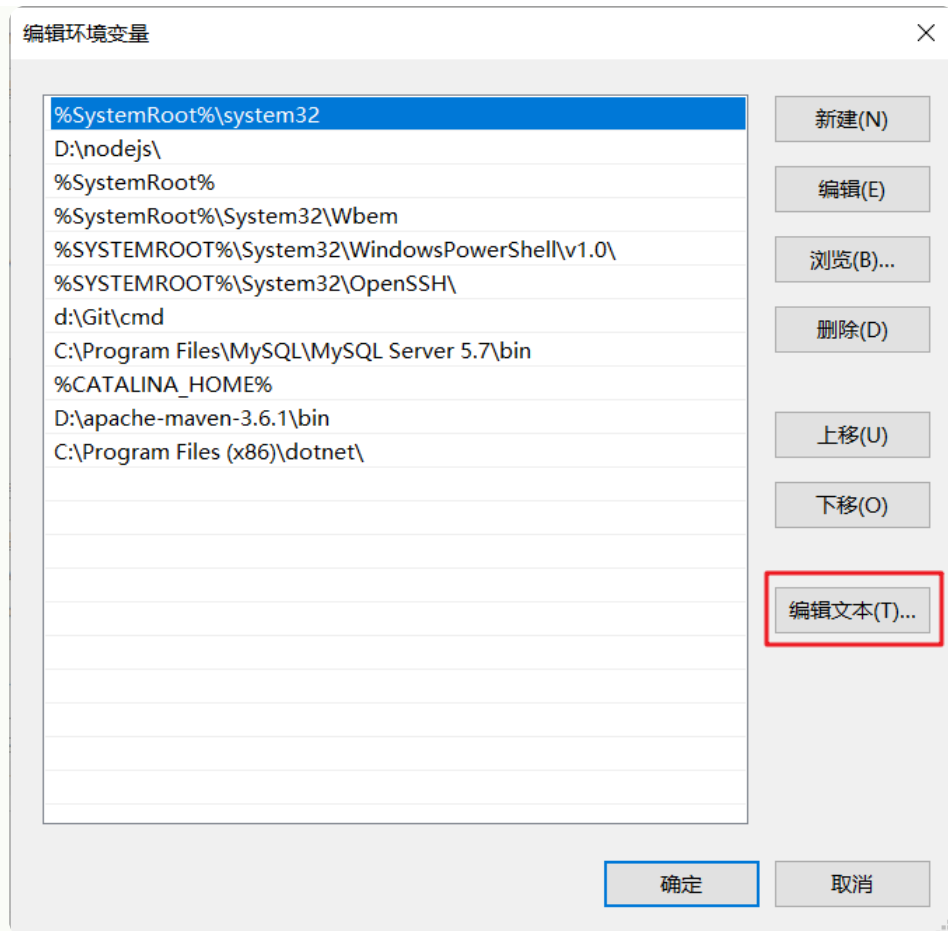
选择环境变量：



找到Path变量，点击编辑：

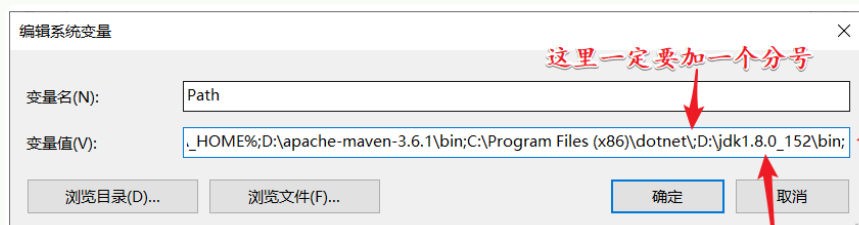


在弹出的对话框选择"编辑文本":



粘贴前面复制的路径:

一定要注意分号



这里一定要加一个分号

最后这个分号可有可无

把刚刚复制的路径粘贴到这里

4.1.3 测试Java环境

打开一个新的cmd窗口（记住，一定要是新的）：输入 `javac` 指令，如果出现下图所示，那么就表示搭建成功

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.19042.985]
(c) Microsoft Corporation。保留所有权利。

C:\Users\Administrator>javac
用法: javac <options> <source files>
其中, 可能的选项包括:
    -g                    生成所有调试信息
    -g:none               不生成任何调试信息
    -g: {lines, vars, source} 只生成某些调试信息
    -nowarn               不生成任何警告
    -verbose              输出有关编译器正在执行的操作的消息
    -deprecation          输出使用已过时的 API 的源位置
    -classpath <路径>    指定查找用户类文件和注释处理程序的位置
    -cp <路径>           指定查找用户类文件和注释处理程序的位置
    -sourcepath <路径>   指定查找输入源文件的位置
    -bootclasspath <路径> 覆盖引导类文件的位置
    -extdirs <目录>      覆盖所安装扩展的位置
    -endorseddirs <目录> 覆盖签名的标准路径的位置
    -proc: {none, only}  控制是否执行注释处理和/或编译。
    -processor <class1>[, <class2>, <class3>...] 要运行的注释处理程序的名称; 绕过
    -processorpath <路径> 指定查找注释处理程序的位置
```

如果是出现 '**javac**' 不是内部或外部命令, 也不是可运行的程序或批处理文件 等提示, 代表环境搭建失败, 如果所示:

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.19042.985]
(c) Microsoft Corporation。保留所有权利。

C:\Users\Administrator>javac
'javac' 不是内部或外部命令, 也不是可运行的程序
或批处理文件。

C:\Users\Administrator>
```

4.2 编写Java代码

4.2.1 编写源代码

Java开发环境已经搭建完毕, 可以开发我们第一个Java程序了。

新建一个HelloWorld.java文件:

```
HelloWorld.java - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

public class HelloWorld{
    public static void main(String[] args){
        System.out.println("2020-05-24: Hello World");
    }
}
```

编写如下代码:

```
1 public class HelloWorld{
2     public static void main(String[] args){
3         System.out.println("2020-05-24: Hello World!!");
4     }
5 }
```

文件名必须是 **HelloWorld**，保证文件名和类的名字是一致的，注意大小写。每个字母和符号必须与示例代码一模一样。

我们的第一个Java源程序就编写完成了，但是这个文件是程序员编写的，JVM是看不懂的，也就不能运行，因此我们必须将编写好的Java源文件，编译成JVM可以看懂的字节码文件。

4.2.2 编译Java程序

在DOS命令行中，进入Java源文件的目录，使用 **javac** 命令进行编译。

格式如下：

javac Java源文件名.后缀名

示例：

javac HelloWorld.java

编译：

C:\Windows\System32\cmd.exe

Microsoft Windows [版本 10.0.19042.985]
(c) Microsoft Corporation。保留所有权利。

D:\code>javac HelloWorld.java

D:\code>

1) 必须进入到D:/code目录

2) 使用javac命令进行编译

运行：


```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.19042.985]
(c) Microsoft Corporation。保留所有权利。

D:\code>javac HelloWorld.java

D:\code>java HelloWorld
2020-05-24: Hello World

D:\code> 1) 运行时不需要加后缀 (.class)
          2) 使用java命令运行
```

4.3 编译和运行

- **编译**：是指将我们编写的Java源文件翻译成JVM认识的class文件，在这个过程中，javac 编译器会检查我们所写的程序是否有错误，有错误就会提示出来，如果没有错误就会编译成功。
- **运行**：是指将 class文件 交给JVM去运行，此时JVM就会去执行我们编写的程序了

关于main方法：main方法：称为主方法。写法是固定格式不可以更改。main方法是程序的入口点或起始点，无论我们编写多少程序，JVM在运行的时候，都会从main方法这里开始执行。

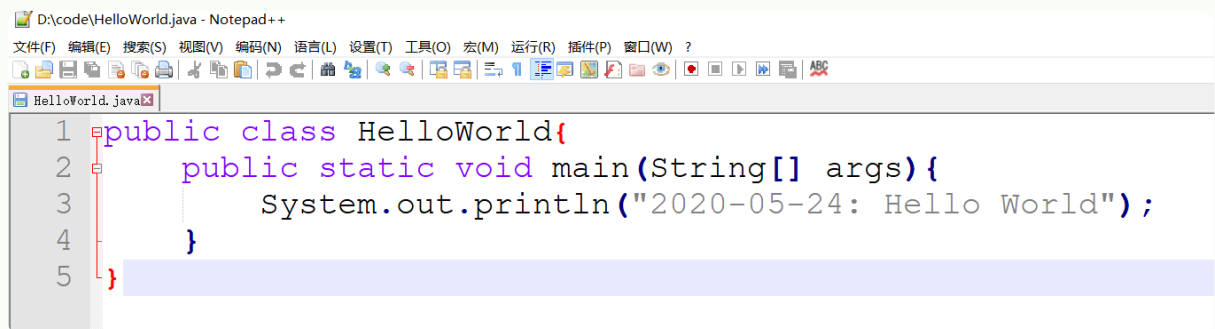
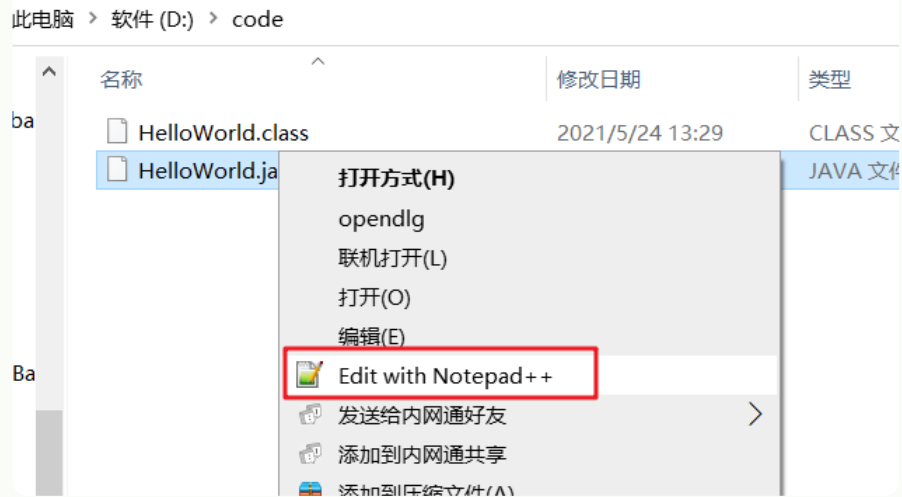
4.4 安装Notepad++高级记事本

Notepad (记事本) 是一款开源、小巧、免费的纯文本编辑器，比Windows自带的记事本要强大很多。Notepad内置支持多达 27 种语法高亮度显示，并提供代码折叠、显示行号等功能；



安装Notepad++的过程非常简单，就和我们平时安装QQ、微信等应用程序一样，这里就不一一截图了；

Notepad安装完毕之后，以后编辑文本可以选择使用Notepad打开：



使用Notepad++打开后的Java程序，提供了关键字高亮、显示行号、还提供代码折叠等功能；

4.5 注释

注释：对代码解释和说明的文字。其目的是让人们能够更加轻松地了解代码。为代码添加注释，是十分必须有的，注释是给人看的，它不影响程序的编译和运行。

- Java中有单行注释和多行注释
 - 单行注释以 `//` 开头 换行结束
 - 多行注释以 `/*` 开头 以 `*/` 结束

示例代码：

```
1 public class HelloWorld{
2     public static void main(String[] args){
3
4         // 我是注释，给程序员看的，JVM不会执行该行代码
5         System.out.println("2020-05-24: Hello World");
6
7         /*
8             我是注释，给程序员看的，JVM不会执行这里面的代码
9             我是注释，给程序员看的，JVM不会执行这里面的代码
10            我是注释，给程序员看的，JVM不会执行这里面的代码
11        */
12     }
13 }
```

The screenshot shows a Notepad++ window with a Java file named 'HelloWorld.java'. The code is as follows:

```
1 public class HelloWorld{
2     public static void main(String[] args){
3
4         // 我是注释，给程序员看的，JVM不会执行该行代码
5         System.out.println("2020-05-24: Hello World");
6
7         /*
8             我是注释，给程序员看的，JVM不会执行这里面的代码
9             我是注释，给程序员看的，JVM不会执行这里面的代码
10            我是注释，给程序员看的，JVM不会执行这里面的代码
11         */
12     }
13 }
```

4.6 关键字

关键字：Java已经定义好的单词，具有特殊含义。

HelloWorld案例中，出现的关键字有 public、class、static、void 等，关键字全部都是 **小写字母**，Notepad++中颜色特殊。

This screenshot is identical to the one above, but with four red arrows pointing to the keywords in the code: 'public' on line 1, 'class' on line 1, 'static' on line 2, and 'void' on line 2.

public、class、static、void等都是关键字，具有特殊含义（功能）

- Java关键字词汇表：

abstract	assert	boolean	break	byte
case	catch	char	class	const
continue	default	do	double	else
enum	extends	final	finally	float
for	goto	if	implements	import
instanceof	int	interface	long	native
new	package	private	protected	public
return	strictfp	short	static	super
switch	synchronized	this	throw	throws
transient	try	void	volatile	while

4.7 标识符（名字）

- 标识符**：是指在程序中，我们自己定义的名字。比如类的名字、方法的名字和变量的名字等等，都是标识符。
 - HelloWorld案例中，出现的标识符有类名字 HelloWorld。
- 命名规则**：**硬性要求**
 - 标识符（名字）可以包含 英文字母26个(区分大小写)、0-9数字、\$（美元符号）和_（下划线）。
 - 标识符不能以数字开头。
 - 标识符不能是关键字**。
 - 区分大小写。
- 命名规范**：**软性建议**
 - 类名规范：每个单词首字母大写（大驼峰式）。
 - 方法名和变量名规范：第一个单词全小写，后面每个单词首字母大写（小驼峰式）。
 - 包名规范：全部小写。

五、数据类型、常量、变量

5.1 数据类型

Java的数据类型分为两大类：

- 基本数据类型**：包括 整数、浮点数、字符、布尔
- 引用数据类型**：包括 类、数组、接口

四类八种基本数据类型：

数据类型	关键字	内存占用	取值范围
字节型	byte	1个字节	-128~127
短整型	short	2个字节	-32768~32767
整型	int（默认）	4个字节	-2147483648~2147483647
长整型	long	8个字节	-2的63次方~2的63次方-1
单精度浮点数	float	4个字节	1.4013E-45~3.4028E+38
双精度浮点数	double（默认）	8个字节	4.9E-324~1.7977E+308
字符型	char	2个字节	0-65535
布尔类型	boolean	1个字节	true, false

Java中的默认类型：整数类型是 int、浮点类型是 double;

5.2 常量

- 常量：是指在Java程序中固定不变的数据。

练习：输出各种类型的常量。

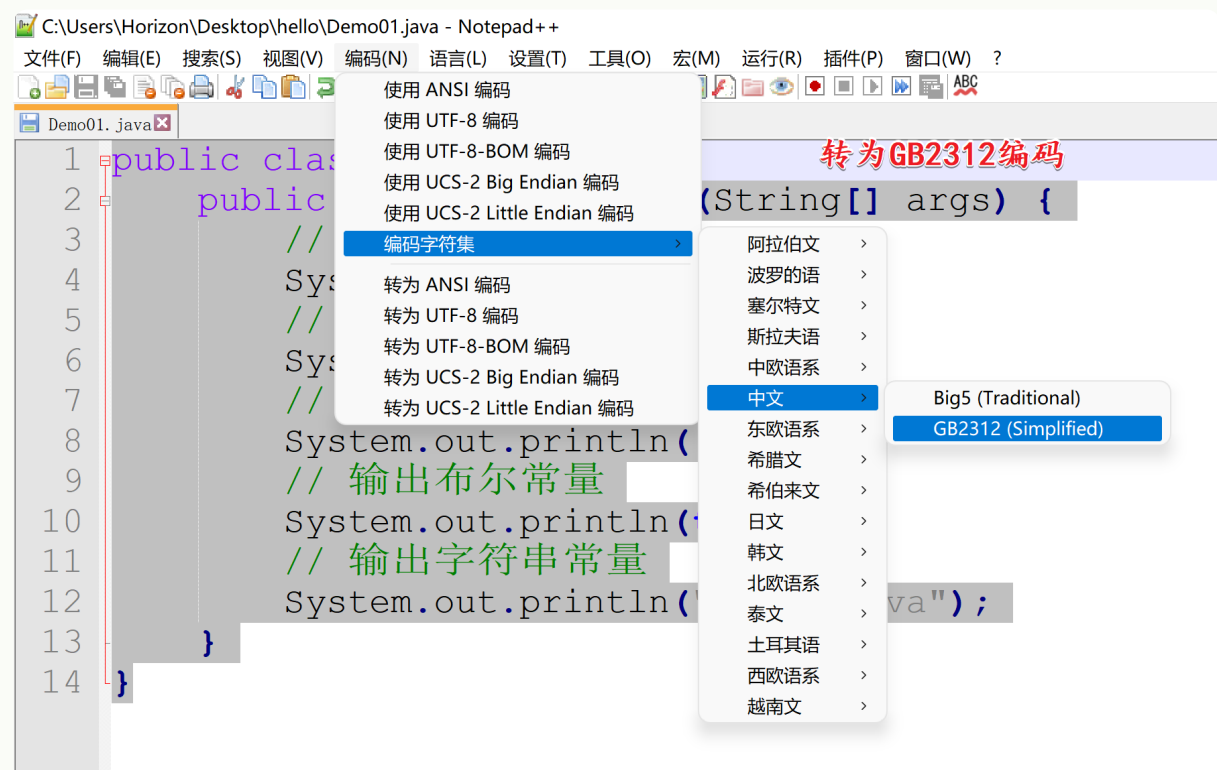
```
1 public class Demo01 {
2     public static void main(String[] args) {
3         // 输出整数常量
4         System.out.println(123);
5         // 输出小数常量
6         System.out.println(0.125);
7         // 输出字符常量
8         System.out.println('A');
9         // 输出布尔常量
10        System.out.println(true);
11        // 输出字符串常量
12        System.out.println("Hello Java");
13    }
14 }
```

编译：

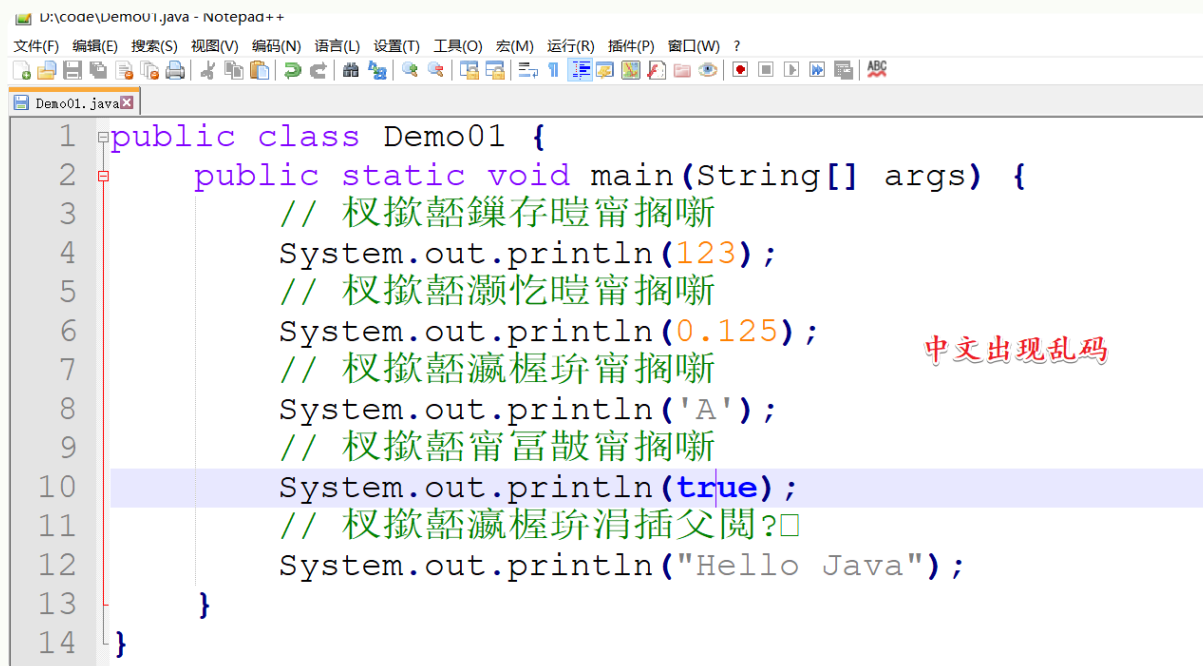
```
D:\code>javac Demo01.java
Demo01.java:11: 错误: 编码GBK的不可映射字符
// 权撒話瀛樞玢涓插父閱?
1 个错误
D:\code>
```

发现出现错误；这个错误是一般是因为代码中含有中文字符，注释中的中文字符也算。由于使用cmd运行java程序的时候，系统默认的编码格式是gbk。而包含中文字符的代码一般是unicode格式，所以直接运行含有中文字符的代码就容易出现编码错误。

进行转码：



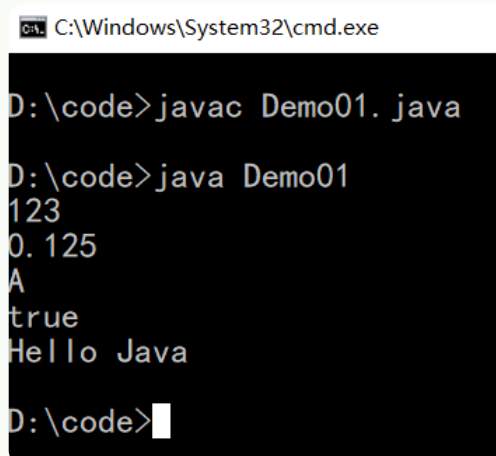
转码之后发现中文都出现乱码了：



```
1 public class Demo01 {
2     public static void main(String[] args) {
3         // 杈嶳髹鐮存曄甯搁噏
4         System.out.println(123);
5         // 杈嶳髹灑屹曄甯搁噏
6         System.out.println(0.125);
7         // 杈嶳髹瀛惲玢甯搁噏
8         System.out.println('A');
9         // 杈嶳髹甯馗馗甯搁噏
10        System.out.println(true);
11        // 杈嶳髹瀛惲玢涓插父閱?
12        System.out.println("Hello Java");
13    }
14}
```

转码之后中文会变成乱码，因此我们以后再进行转换之前最后先复制一份；

再次编译运行：



```
C:\Windows\System32\cmd.exe

D:\code>javac Demo01.java

D:\code>java Demo01
123
0.125
A
true
Hello Java

D:\code>
```

5.3 变量

- 变量：变量是内存中的一小块区域，其值可以在一定范围内变化

数学中，可以使用字母代替数字运算，例如 $x=1+5$ 或者 $6=x+5$ 。

程序中，可以使用字母保存数字的方式进行运算，提高计算能力，可以解决更多的问题。比如x保存5，x也可以保存6，这样x保存的数据是可以改变的，也就是我们所讲解的变量。

Java中要求一个变量每次只能保存一个数据，必须要明确保存的数据类型。

5.4 变量的定义

变量定义的格式包括三个要素：数据类型、变量名、数据值。

练习：定义所有基本数据类型的变量，代码如下：

```
1 public class Demo02 {
2     public static void main(String[] args) {
3         //定义字节型变量
4         byte b = 100;
5         System.out.println(b);
6         //定义短整型变量
7         short s = 1000;
8         System.out.println(s);
9         //定义整型变量
10        int i = 123456;
11        System.out.println(i);
12        //定义长整型变量
13        long l = 12345678900L;
14        System.out.println(l);
15        //定义单精度浮点型变量
16        float f = 5.5F;
17        System.out.println(f);
18        //定义双精度浮点型变量
19        double d = 8.5D;
20        System.out.println(d);
21        //定义布尔型变量
22        boolean bool = false;
23        System.out.println(bool);
24        //定义字符型变量
25        char c = 'A';
26        System.out.println(c);
27    }
28 }
```

long类型：建议数据后加L表示。

float类型：数值后面必须加上F

double类型：建议数据后加D表示。

为了规范起见，我们还是统一都加上L/F/D

编译运行：

```
C:\Windows\System32\cmd.exe

D:\code>javac Demo02.java

D:\code>java Demo02
100
1000
123456
12345678900
5.5
8.5
false
A
D:\code>
```

5.5 定义变量注意事项

1. 定义的变量，不赋值不能使用。
2. 同一个方法中变量名不能重复。
3. 变量只在定义时的{}中有效。

```

1 public class Demo2 {
2
3     public static void main(String[] args) {
4         int a = 10;
5         System.out.println(a);
6
7         int b;
8         b = 20; // 变量在使用之前一定要赋值
9         System.out.println(b); // 错误: 可能尚未初始化变量b
10
11         // double a = 3.1; // 错误: 已在方法 main(String[])中定义了变量 a
12         // System.out.println(a);
13         double c = 3.1; // 错误: 已在方法 main(String[])中定义了变量 a
14         System.out.println(c);
15
16         {
17             int d = 40;
18             System.out.println(d);
19         }
20
21         // System.out.println(d); // 错误: 找不到符号
22     }
23 }

```

5.6 String类型

String类型是一种引用数据类型，用于存储字符串（多个字符），我们知道char类型只能存储单个字符，不能够存储多个字符，而String就是用于存储多个字符串的；

关于什么是引用数据类型我们在后面章节进行讲解；

String类型使用：

```

1 public class Demo3 {
2     public static void main(String[] args) {
3
4         String str1="hello~";
5
6         String str2="你好啊~";
7
8         String str3="嘿嘿~";
9
10        System.out.println(str1);
11        System.out.println(str2);
12        System.out.println(str3);
13    }
14 }

```